Sorting

CMPT. 225

SPRING 2024.

Sorting

- re-order elements of a sequence*

  S s.t. $s_0 \leq s_1 \leq s_2 \leq \cdots \leq s_{n-1}$ **

- We will look at 5 sorting algorithms:

  - 3 iterative // they repeat some step

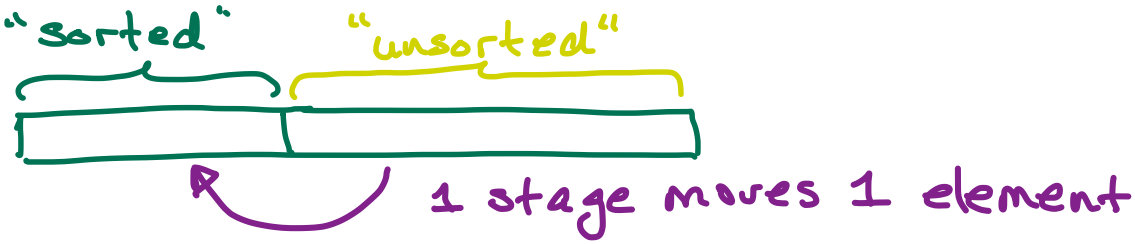
  - 2 recursive // they are divide-and-conquer

* we typically assume they are in an array, but this need not be.

** so the elements are from some ordered set.

The iterative algorithms:

- maintain a partition: "unsorted part" & "sorted part"
- sort a sequence of n elements in n-1 stages
- at each stage, move 1 element from the "unsorted part" to the "sorted part":

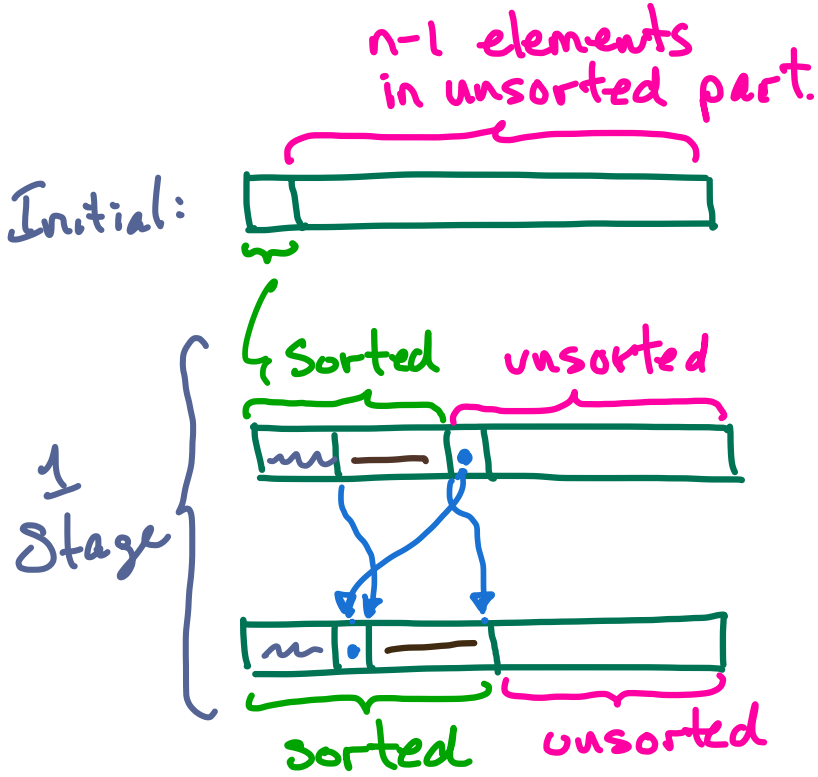



```
sort(A){
    · initialize
    · repeat n-1 times
        move 1 element from unsorted to sorted part
}       // unsorted part gets smaller; sorted part gets larger.
```

- the algorithms differ in how they:
  - select an element to remove from the unsorted part.
  - insert it into the sorted part.

Insertion Sort

- initially: sorted part is just A[0]
- repeat n-1 times:
  - remove the first element from the unsorted part
  - insert it into the sorted part (shifting elements to the right as needed)



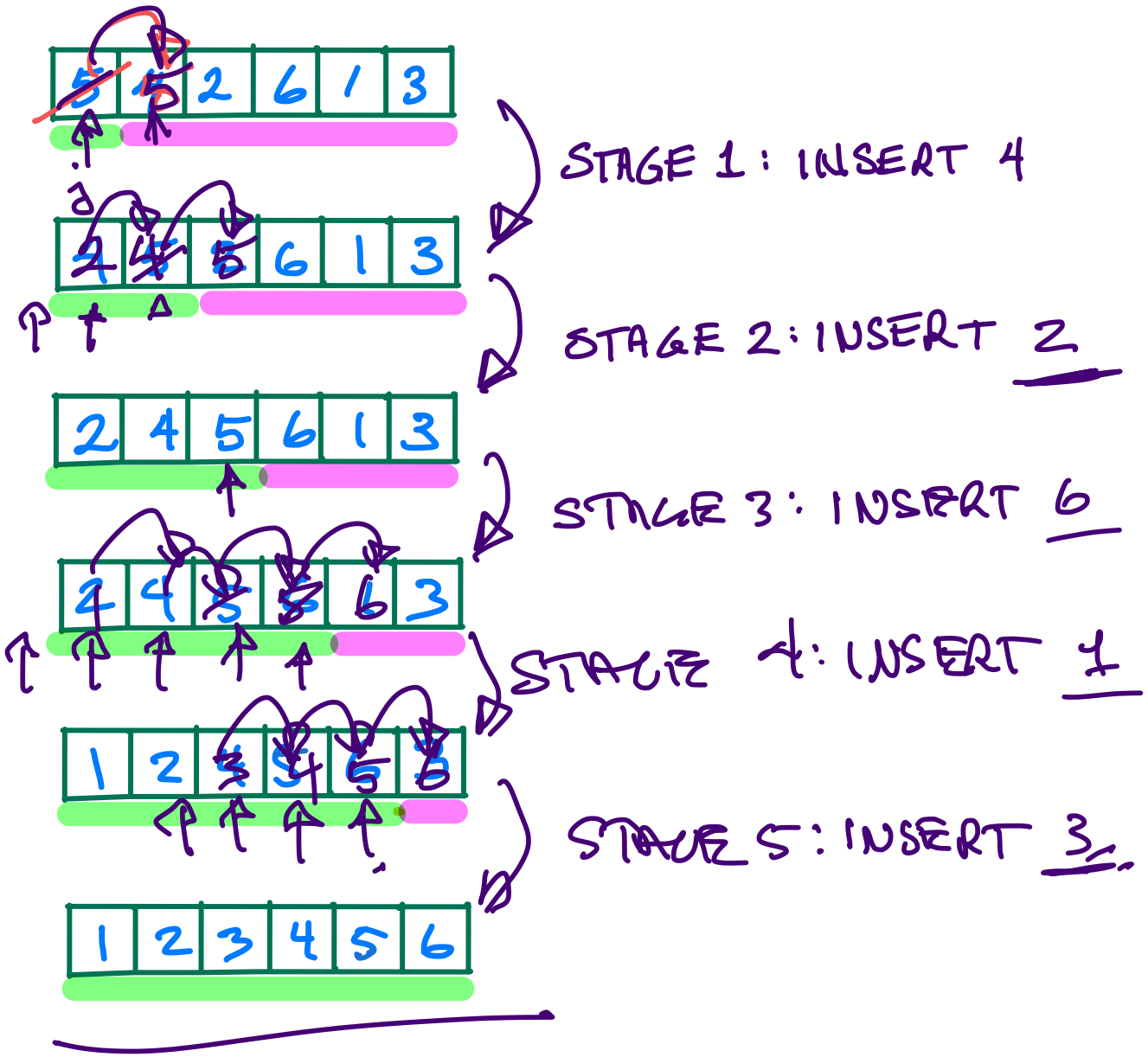

```
insertion_sort(A)
    for( i = 1 to n-1 ){
        pivot = A[i] // first element in unsorted
                     // part
        j = i - 1
        while(j ≥ 0  AND A[j] > pivot){
            A[j+1] = A[j] // shift jth element
            j = j-1
        }
        A[j+1] = pivot // move pivot into position.
    }
```

Shift all elements in sorted part that are larger than pivot 1 "to the right".

Insertion Sort Example

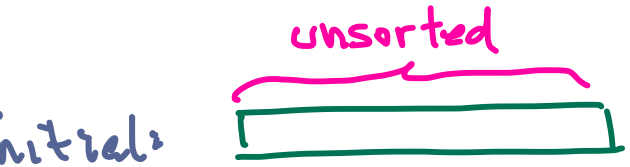
Sorted Part

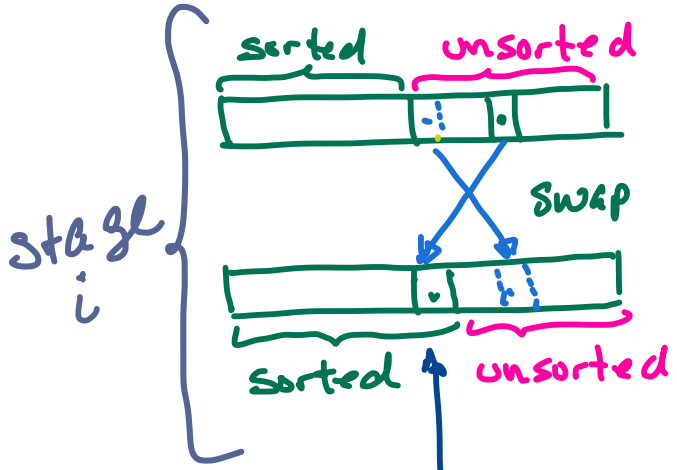
Unsorted Part



Selection Sort

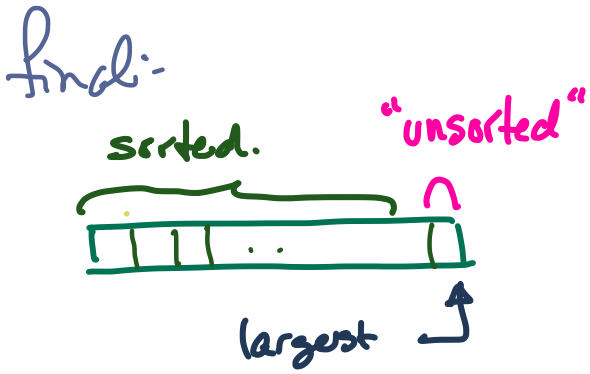
- initially, sorted part empty
- repeat n-1 times
  - find a smallest element in the unsorted part (by sequential search)
  - swap it into the first position of the unsorted part, which becomes the new last position of sorted part.



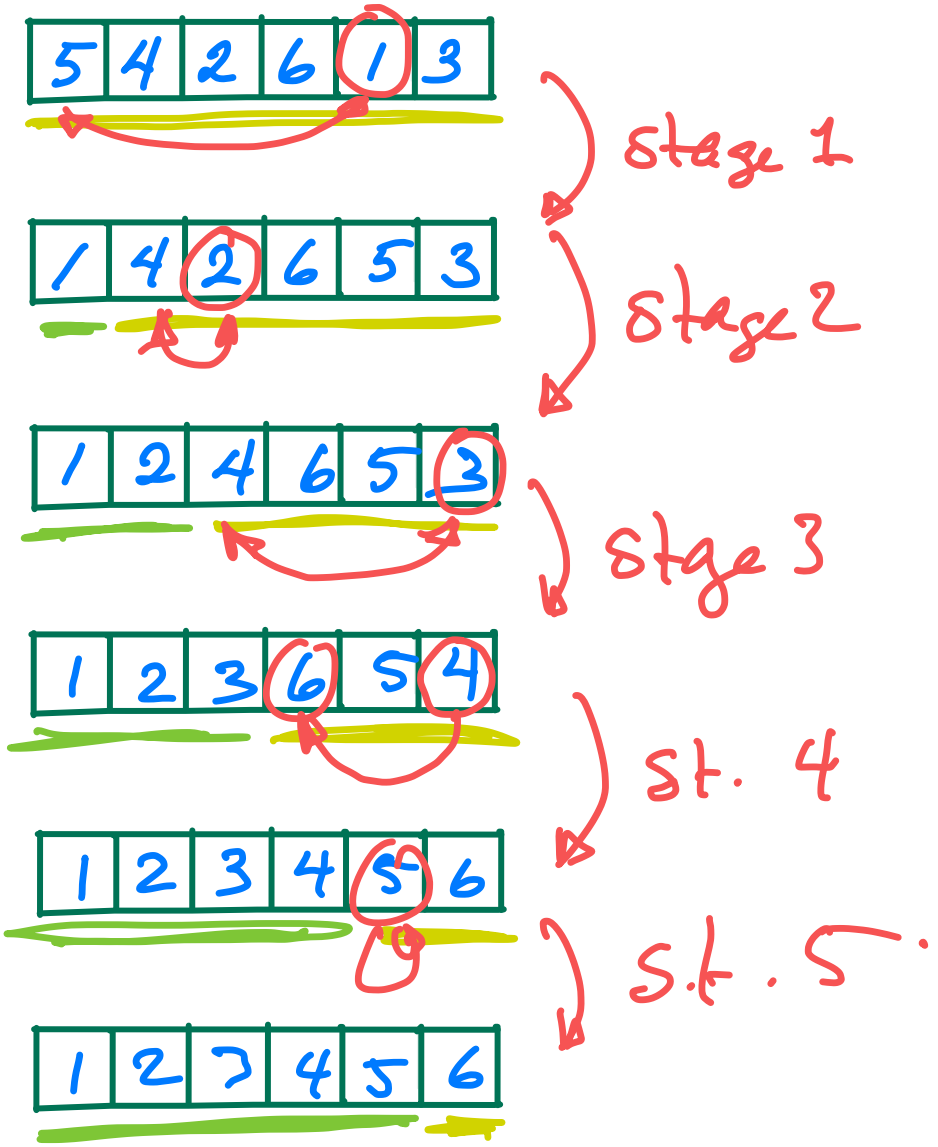



```
selection_sort(A){
  for( i=1 to n-1 ){
    j= i-1  // j is index of min found so far.
    k=i
    while( k<n){
      if( A[k] <A[j]) j=k;
      k = k+1
    }
    //j is index of min. element
    swap A[i-1] and A[j]
  }
}
```

find min element in unsorted part



Selection Sort Example



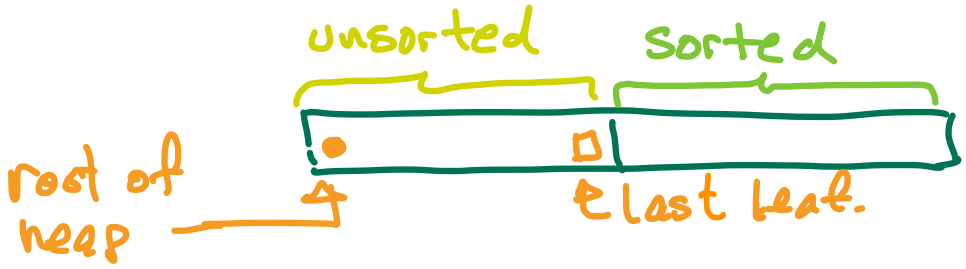
- Selection sort repeatedly finds a smallest element by search.
- A heap does this without search, in time $O(\log n)$ rather than $\Theta(n)$.

~~Selection Sort~~ Heapsort (idea)

- initially, sorted part empty
- make unsorted part into a heap ← takes $O(n)$ time
- repeat $n-1$ times
  - find the smallest element in the unsorted part } heap extract takes $\log(n)$ time (vs. $\Theta(n)$ for the scan in Selection Sort).
  - move it to the first position which becomes the new last position of sorted part.

Heapsort: Arrangement in Array.

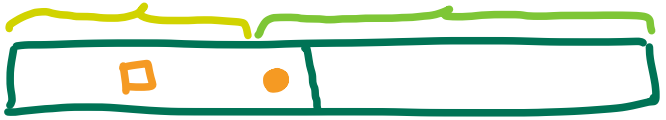
We use a max-heap, and this arrangement:



So: • the heap root is at A[0]

• heap extraction removes •, moves □ to A[0], freeing up the spot where • belongs.

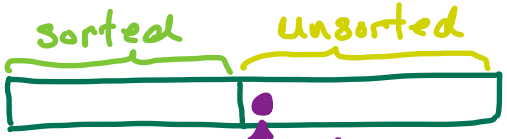
Leaving us:



(Re-coding a min heap into a max heap is just replacing < with > and vice versa.)

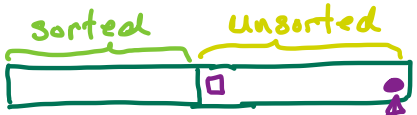
Other possible arrangements in the array:

(1)



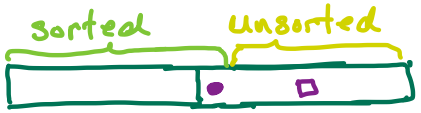
if this is the root of the heap, then it is also the smallest element in the unsorted part, so is in its correct final position. To use this arrangement, the root of the heap keeps moving, so we have lots of shifting to do.

(2)



If this is the root of the heap, then everything works:

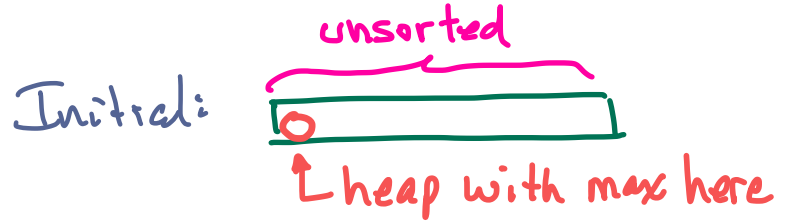
- we extract •; move the last leaf □ to the root + do a percolate-down; store • where □ <u>was</u>, which is now free, and is the correct final location for •; after which we have:

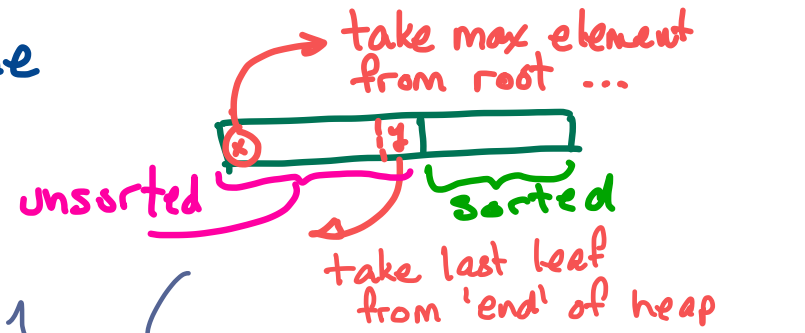


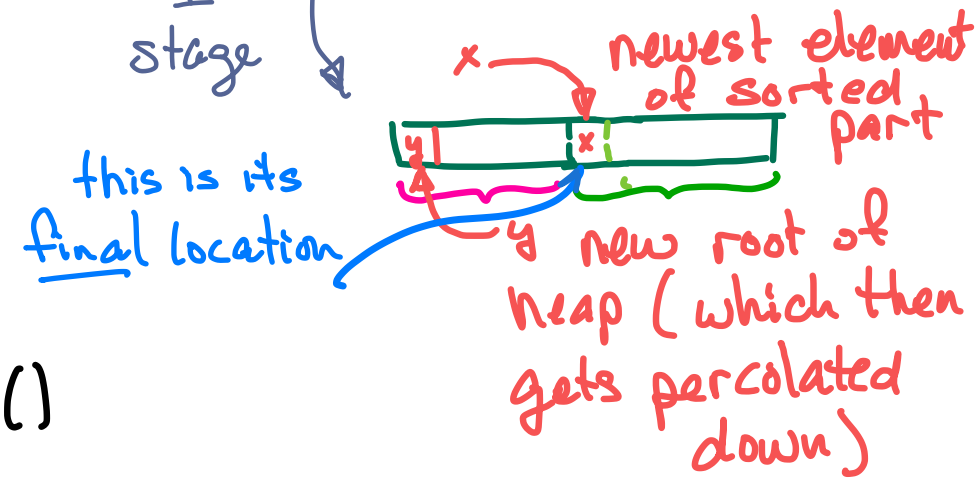
<u>But</u>: we must re-code our heap implementation s.t. the root is at A[n-1], with the result that the indexing is now less intuitive.

Heapsort

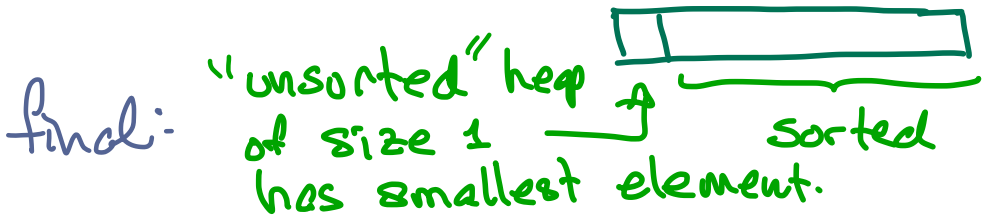
- initially, sorted part empty
- make unsorted part into a max heap
- repeat n-1 times
  - extract a largest element from the heap (unsorted part)
  - move it to the new first position in the sorted part.








```
heapsort(A){
  buildMaxheap(A)
  for( i=1 to n-1 ){
        A[n-i] = extractMax()
  }
}
```

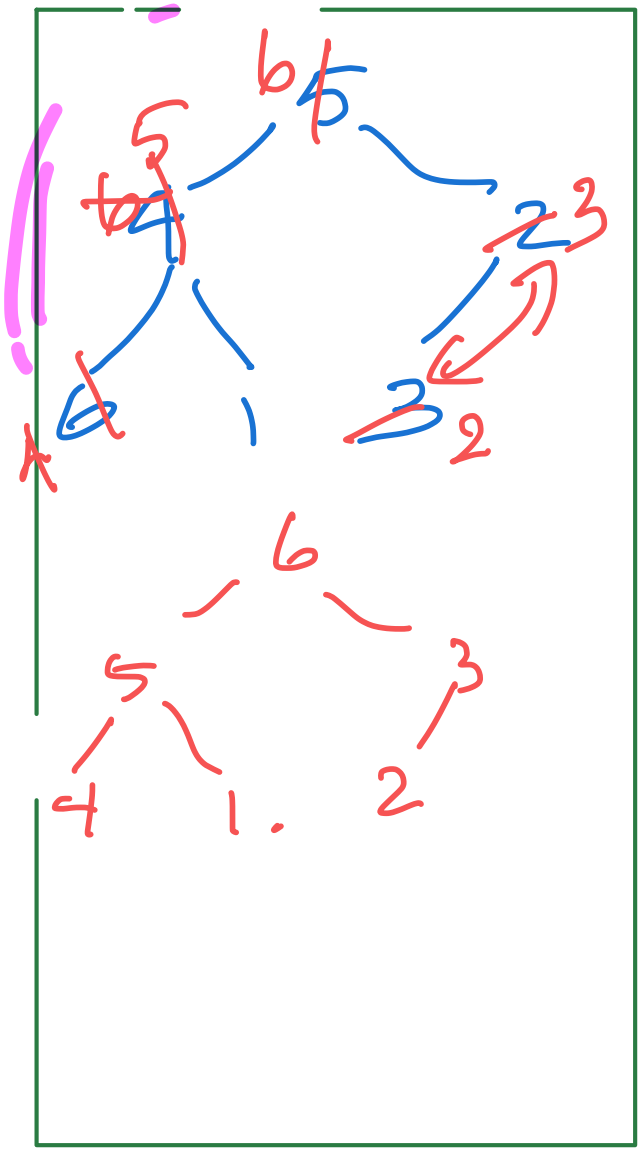


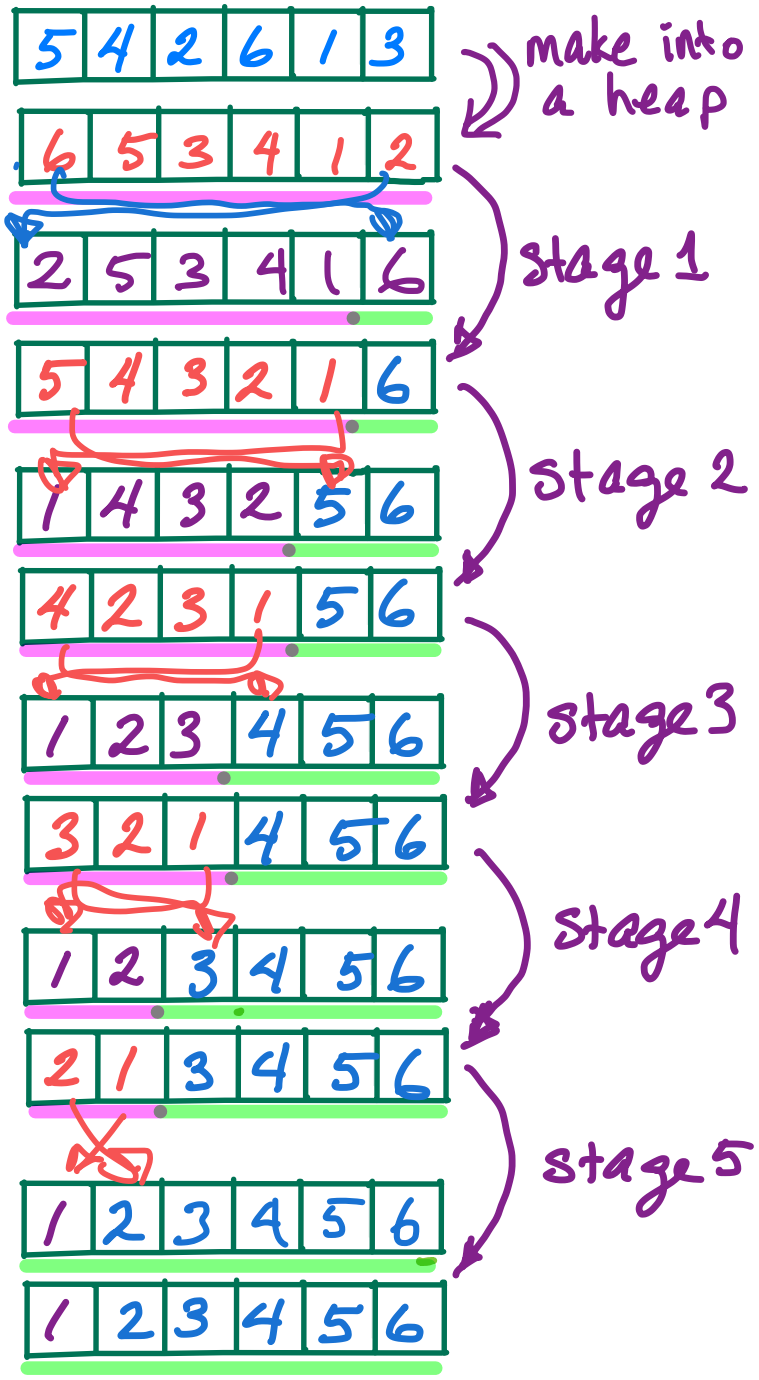
Heapsort

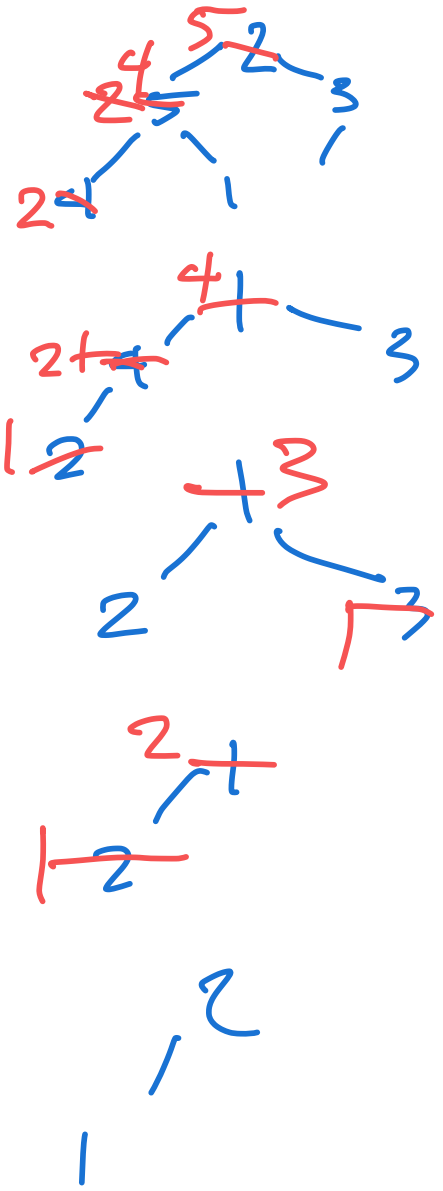
- initially, sorted part empty
- make unsorted part into a max heap
- repeat n-1 times
  - swap the root and last leaf of heap.
    (moves a largest element in the unsorted part to the last location which the becomes the new first location of the sorted part)
  - do a percolate-down at the root to restore heap ordering.

```
heapsort(A){
  make A a max heap
  for i=1.. n-1 {
    swap A[0], A[n-1]
    j ← 0
    while 2j+1 < size {
      child ← 2j+1
      if (2j+2 < size
            & A[2j+2] < A[2j+1]){
        child ← 2j+2
      }
      if (A[child] < A[j]){
        swap A[child], A[j]
        j ← child
      }else
        j ← size
      }
    }
  }
```

Heapsort Example:







Time Complexity of Iterative Sorting Algorithms

- each algorithm does exactly n-1 stages
- the work done at the i^{th} stage varies with the algorithm (& input)
- we take # of item comparisons as a measure of work/time.*

SelectionSort - exactly n-i comparisons to find min. element in unsorted part

InsertionSort - between 1 and i comparisons to find new location for pivot

HeapSort: - between 1 and $2\log_2(n-i+1)$ comparisons for percolate-down

Selection Sort

On input of size n, # of comparisons is always (regardless of input):

$$\sum_{i=1}^{n-1}(n-i) = \sum_{i=1}^{n-1} i$$

$$= S(n-1) \quad \left(\text{Recall that } S(n) = \frac{n(n+1)}{2}\right)$$

$$= \frac{(n-1)(n)}{2}$$

$$= \frac{n^2-n}{2}$$

$$= \Theta(n^2)$$

$$\frac{n^2}{2} - \frac{n}{2}$$

Exercise: Find $c_1, c_2, n_0 > 0$ s.t. $n > n_0 \Rightarrow c_1 \cdot n^2 \leq \frac{n^2-n}{2} \leq c_2 \cdot n^2$

i.e., prove that $\frac{n^2-n}{2} = \Theta(n^2)$

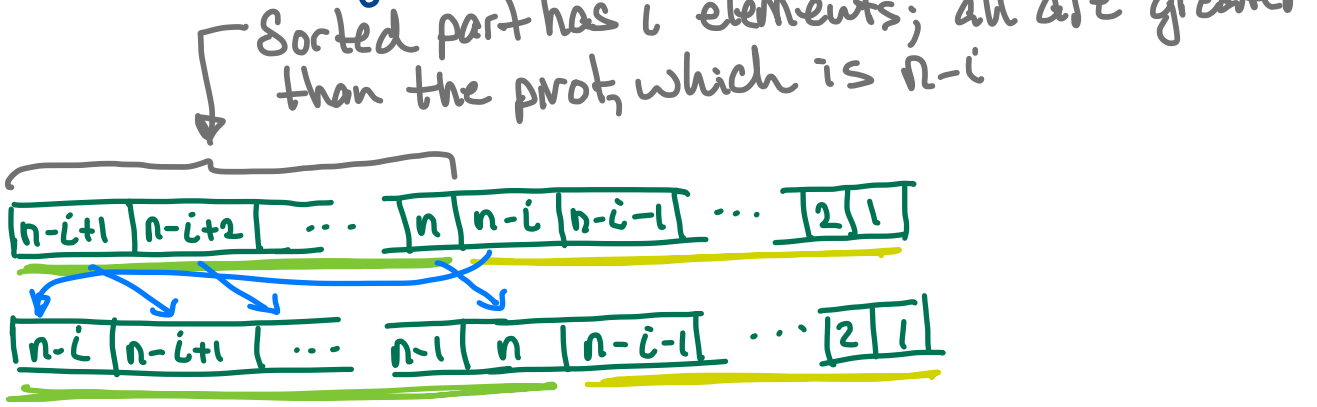
Insertion Sort – Worst Case

Upper Bound: $\#\text{comparisons} \leq \sum_{i=1}^{n-1} i = \frac{n^2-n}{2} = O(n^2)$.

Lower Bound: Worst case: initial sequence is in reverse order.

Eg. 

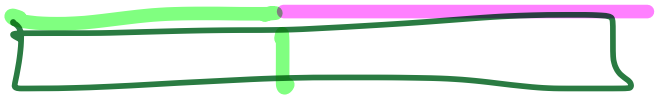
In the i^{th} stage we have:



So, $\#\text{comparisons} \geq \sum_{i=1}^{n-1} i = \Omega(n^2)$

So, Insertion Sort Worst Case is $\Theta(n^2)$

Insertion Sort Best Case



Best case: initial sequence is fully ordered.

Then: In each stage exactly 1 comparison is made.

So: # comparisons $= n-1 = \Theta(n)$.

$\Rightarrow$ Insertion Sort vs. Selection Sort (?)

↑ Better if input mostly in order

↑ Better otherwise, because of # of assignments

Heapsort Worst Case

Upper Bound: # comparisons $\leq \sum_{i=1}^{n-1} 2 \log_2 (n-i+1)$

$$= 2 \sum_{i=1}^{n-1} \log_2 (n-i+1)$$

$$\leq 2 \sum_{i=1}^{n-1} \log_2 n$$

$$\leq 2n \log_2 n$$

$$= O(n \log n)$$

Lower Bound: $\Omega(n)$ is easy – The algorithm does extract-min $n-1$ times, and each extract-min takes does at least 1 swap, so takes $\Omega(1)$ time.

Ex: Prove that if $f(n) = n-1$, then $f(n) = \Theta(n)$.

Best Case? (What input would lead to no movement during percolate-down? What if we exclude this case?)

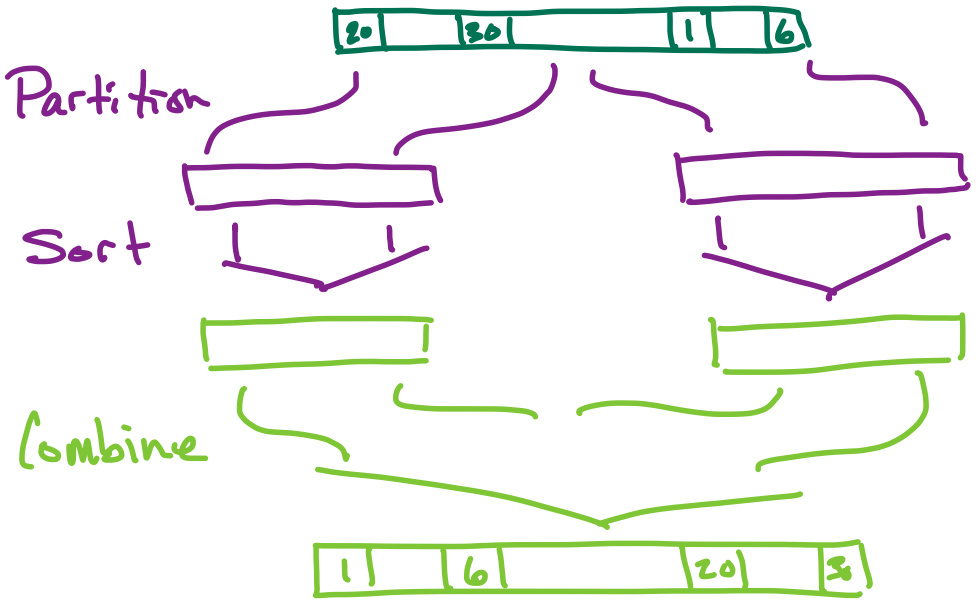
* Number of comparisons

- We must verify #comparisons (or some constant times #comparisons) is an upper bound on work done by each algorithm.

- #of assignments (& swaps) also matters in actual run time. (eg. Insertion sort tends to do many more assignments than Selection Sort).

Recursive Divide & Conquer Sorting

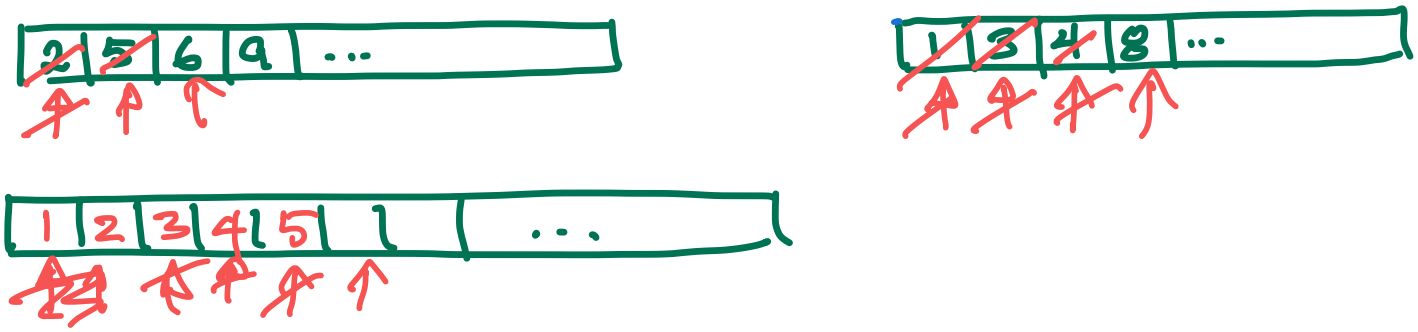
- Partition the sequence A into two parts A_1, A_2
- Recursively sort each of A_1 and A_2
- Combine the sorted versions of A_1 and A_2 to obtain a sorted version of A



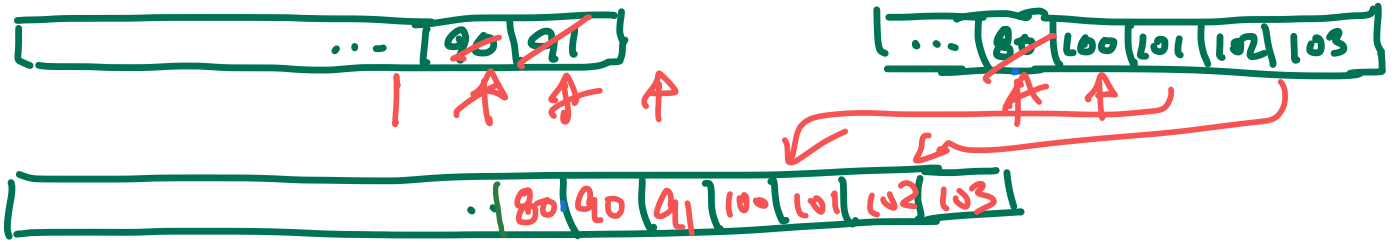
- The algorithms differ in how they choose the partition, and how they combine the sorted parts

Mergesort:

- Uses the fact that merging two sorted lists is easy:



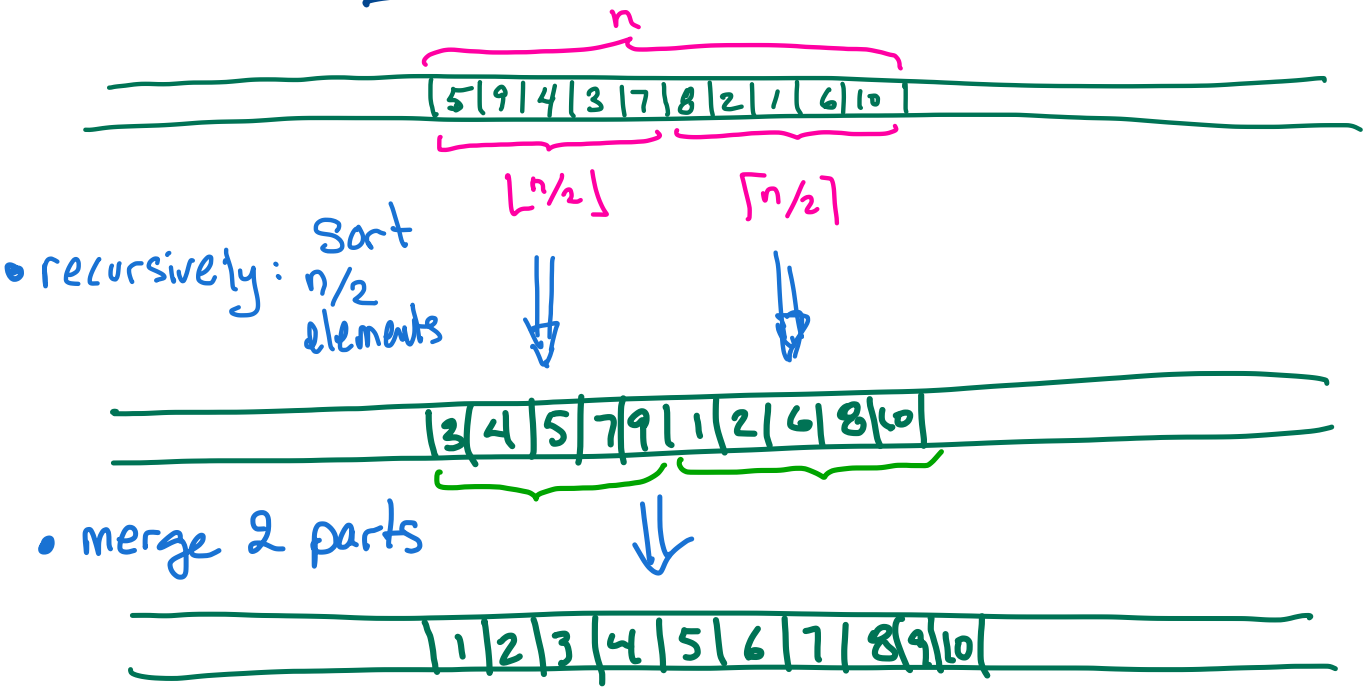
End:



- Takes $\Theta(n)$ time, where n is the total size

Mergesort:

- partition: first half & second half.
- combine: merge the parts



- Works with linked-list or array implementations
- in array implementations, uses $\Theta(n)$ extra space

Mergesort

```
mergesort(A, lo, hi){
    if(lo < hi){ // there are ≥ 2 items, so work to do
        mid ← ⌊(lo + hi)/2⌋
        mergesort(A, lo, mid)
        mergesort (A, mid+1, hi)
        merge (A, low, mid, hi)
    }
}
```

Top level call is mergesort(A, 0, n-1).

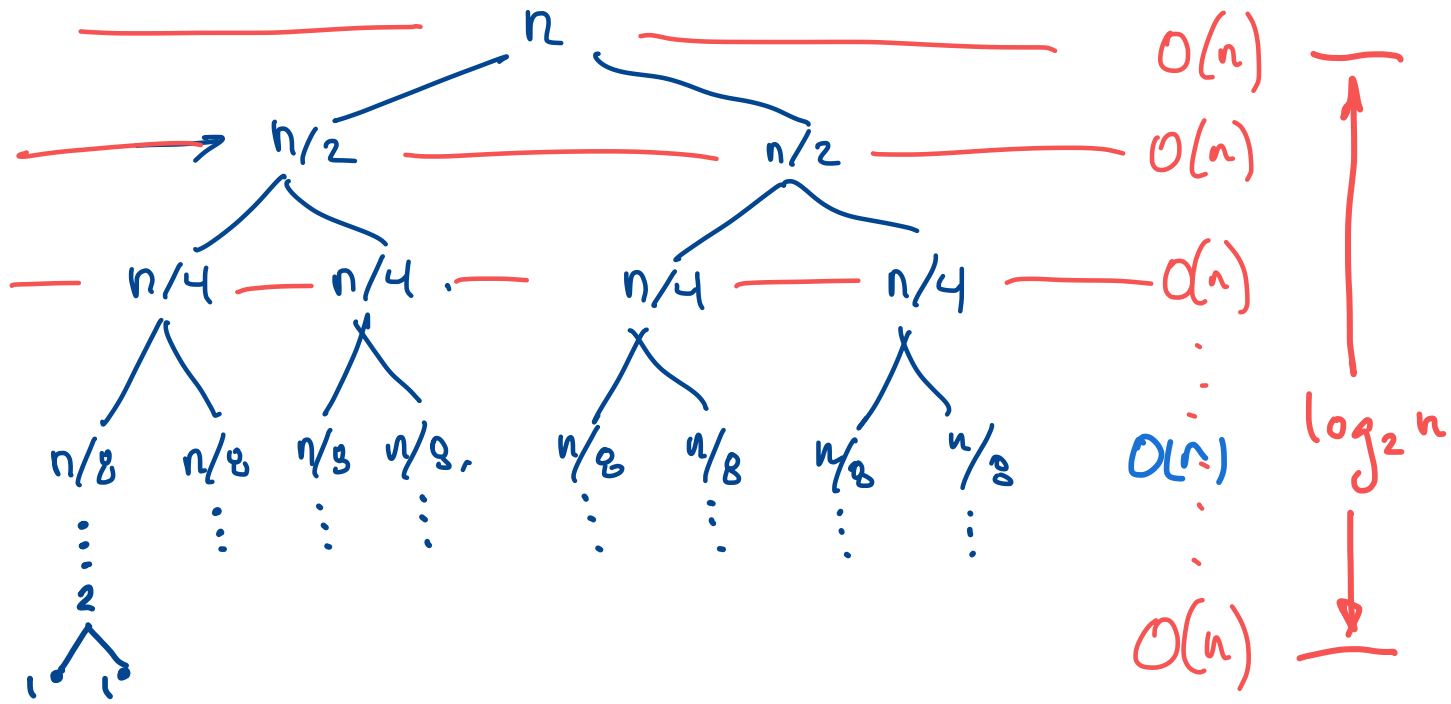
Merge for Merge Sort

(This is more compact than the one shown in lecture, but does the same amount of work.)

```
merge(A, lo, mid, hi){
    left = low;
    right = mid+1;
    for(i = lo; i < hi; i++){ // i ranges over locations in B
        if (left > mid){ // left list is empty, so copy from right list
            B[i] = A[right];
            right++;
        } else if (right > hi){ // right list is empty
            B[i] = A[left];
            left++;
        } else if (A[left] ≤ A[right]){ // left list has smallest remaining element
            B[i] = A[left];
            left++;
        } else
            B[i] = A[right]; // right list has smallest remaining element.
            right++;
        }
    }
    for(i=lo; i<hi; i++) A[i] = B[i]; // copy back from B to A.
}   // there are schemes to avoid this copying.
```

// B is an auxiliary array

Time Complexity of Merge Sort: via tree of recursive calls



- If n is a power of 2, the tree of recursive calls is a perfect binary tree with n leaves, and height $\log_2 n$.
- At depth i there are 2^i calls to merge, each to merge two lists of size $n/2^{i+1}$ into one of size $n/2^i$.

Total work at depth i is $2^i \cdot O(n/2^i) = O(n \, 2^i/2^i) = O(n)$.

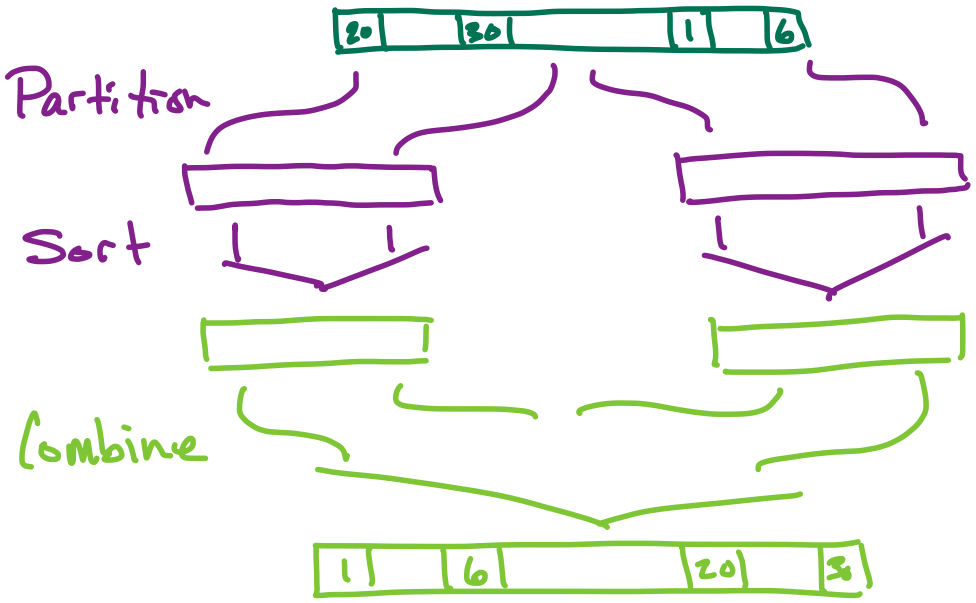
Total work is #depths $\cdot\, O(n) = \log n \cdot O(n) = O(n \log n)$.

Iterative Merge Sort

```
// k = size of blocks. Alg. merges pairs of size-k blocks.
// i = counter. Alg. is merging the ith pair of blocks.
// u = upper end of upper size-k block.
MergeSort( A ){
    k=1
    while ( k<n ){
        i=1
        while( i+k ≤ n ){
            u = i + k*2
            if( u>n+1 ){ u = n+1; }
            merge( A, i, i+k, u )
            i = i + k*2
        }
        k=k*2
    }
}
```

Recursive Divide & Conquer Sorting

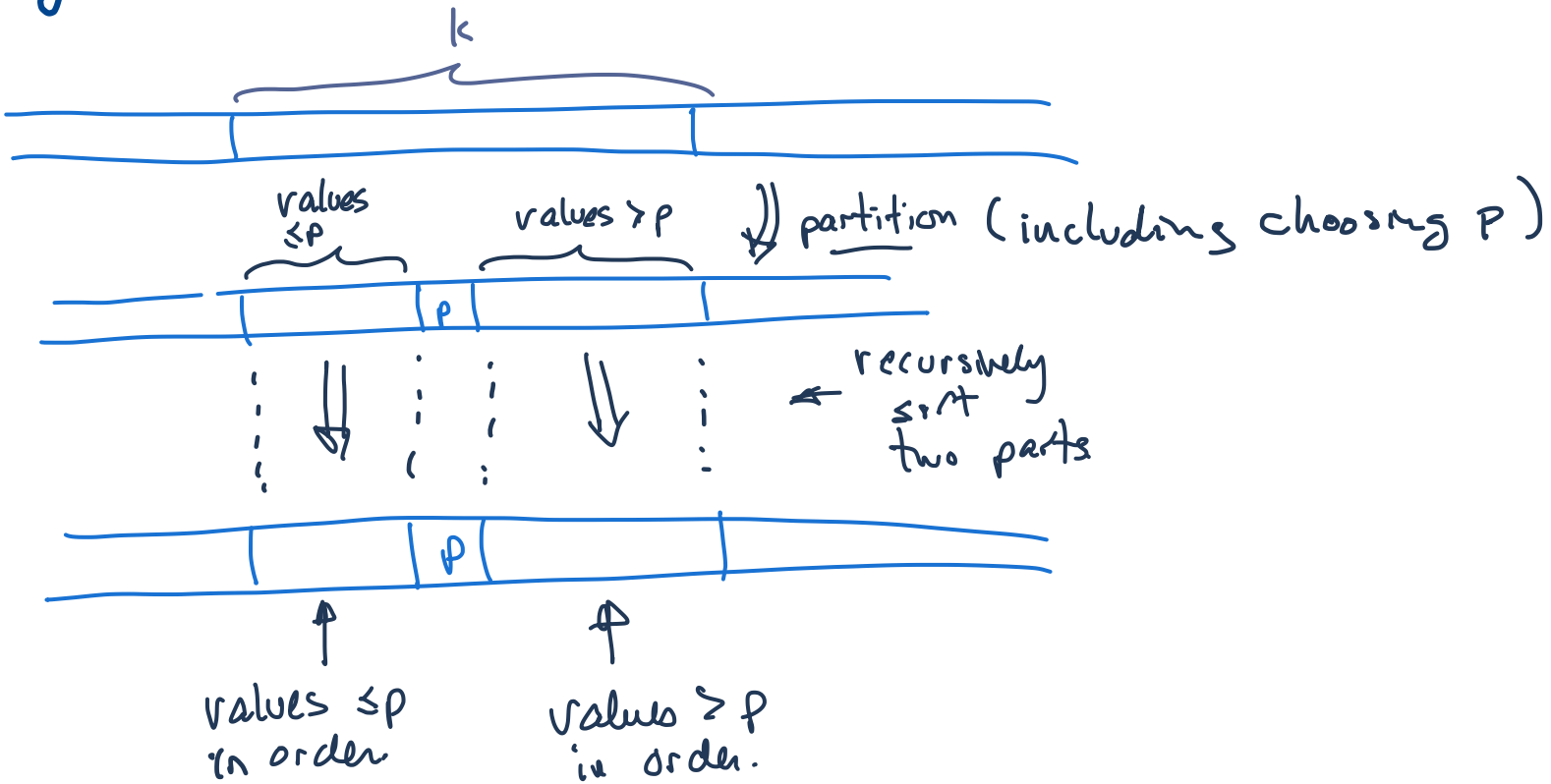
- Partition the sequence A into two parts A_1, A_2
- Recursively sort each of A_1 and A_2
- Combine the sorted versions of A_1 and A_2 to obtain a sorted version of A



- The algorithms differ in how they choose the partition, and how they combine the sorted parts

Quicksort

- Uses a pivot p to partition sequence into "small" and "large" elements: "small elements" $\leq p <$ large elements
- combining sorted versions is trivial



- choosing pivots is key to performance.

Quicksort

```
quicksort(A, lo, hi){
    if(lo < hi){ // there are ≥ 2 items
        pivotposition ← partition(A, lo, hi) // partition
        quicksort(A, lo, pivotposition - 1)
        quicksort(A, pivotposition + 1, hi)
    }
}
```

- Quicksort is correct as long as every call to partition() returns and leaves the variables satisfying the following:

  for every i, j with $lo \le i \le pivotposition \le j \le hi$

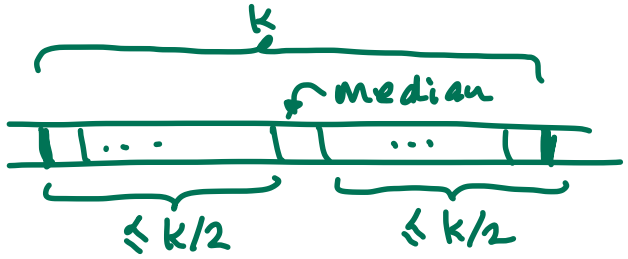
  $$A[i] \le A[pivotposition] \le A[hi]$$

  (One case is: $A[lo] \le A[pivotposition] \le A[hi]$)

- However, efficiency relies critically on good pivot choices.

Ex: Perfect Pivots

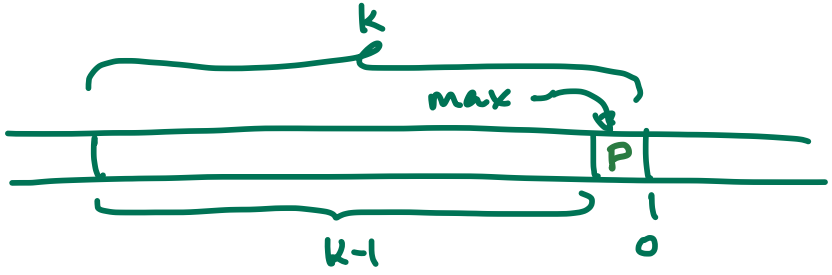
- Suppose all elements are distinct, and the pivot is always the median element in A[lo]... A[hi].
- Then, every call to Quicksort on sequence of size $k \geq 2$ makes two recursive calls on sequences of size $\leq k/2$:



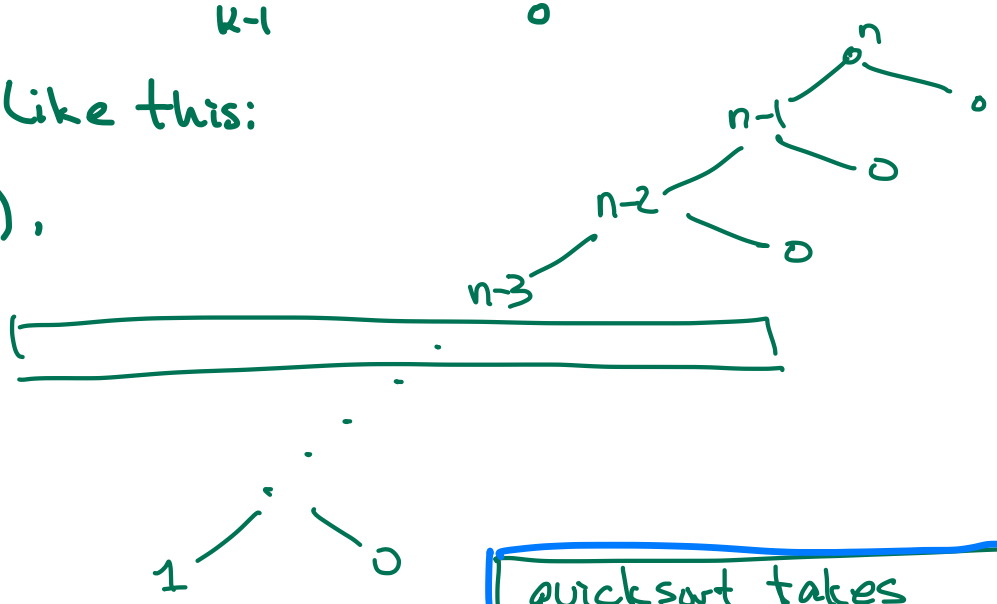
- By essentially the same argument as used for MergeSort, this gives us running time of $\log(n) \cdot f(n)$, where $f(n)$ is the time to run partition on a sequence of size n.
- Assuming $O(n)$ time for partition, this would give us $O(n \log n)$ time for Quicksort.
- But: finding medians is too slow in practice
- Optional exercise: Can the median be found in $O(n)$ time?

Ex: Worst Case Pivots.

- Suppose all elements are distinct, and the max (or min.) is always chosen as pivot.
- Then every call to Quicksort on a sequence of $k \geq 2$ elements makes one recursive call on a sequence of size $k-1$, and one on a sequence of size 0:



- The recursion tree looks like this:
  - This tree is of height $\Theta(n)$, giving us a running time of $\Theta(n) \cdot f(n)$, or $\Theta(n^2)$ assuming $\Theta(n)$ for partition.

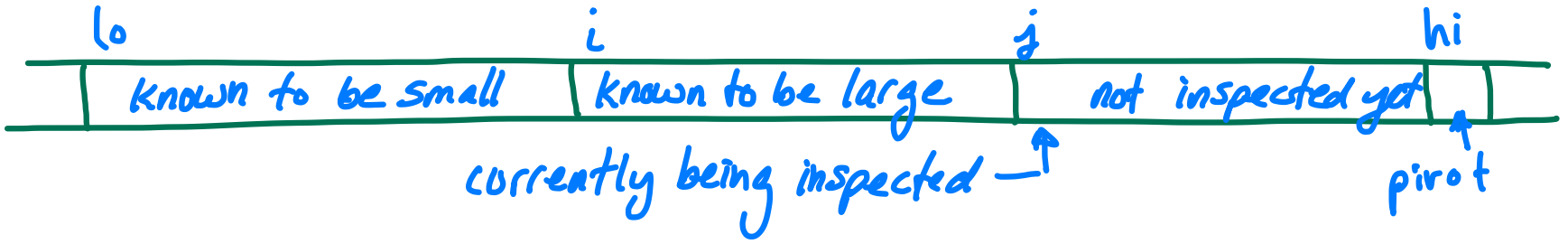


quicksort takes time $\Theta(n^2)$ in the worst case.

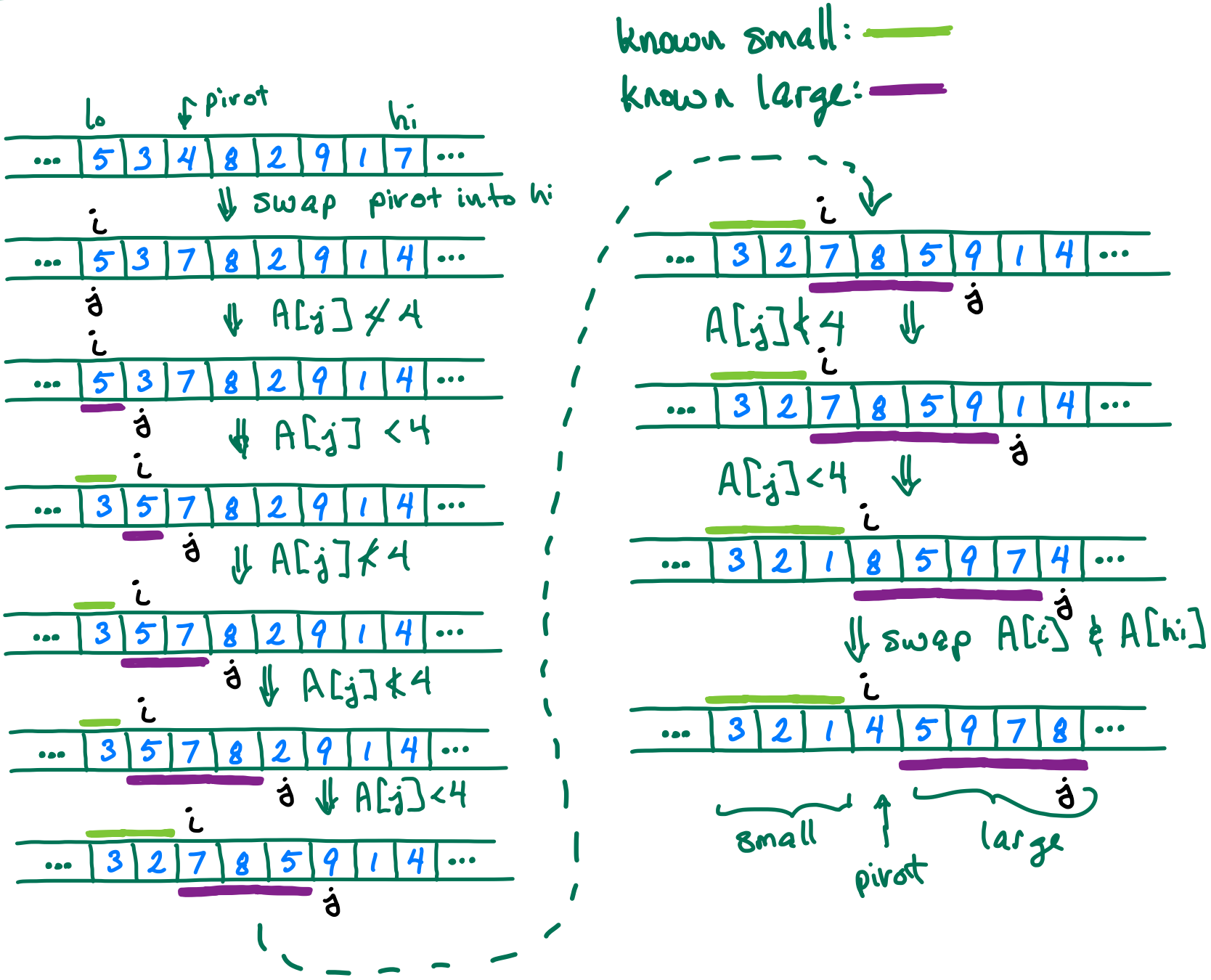
Partition – Simple Version

- Partition must choose a pivot p, and efficiently re-arrange elements

```
partition(A, lo, hi){
    pivotIndex ← choosePivot(A, lo, hi) // choose pivot
    swap A[pivotIndex] and A[hi] // move pivot out of the way.
    p ← A[hi] // p is the pivot value
    i ← lo // known "small" values will be at indices < i
    for(j = lo ; j < hi ; j++){ // "already inspected" values will
                                //be at indices < j
        if (A[j] < p){          // if we are inspecting a "small"
            swap A[i] and A[j]  // swap it with first "non small"
            i ← i+1             // increase size of "smalls" part.
        }
    }
    swap A[i] and A[hi] //move pivot where it belongs.
    return i // this is pivot position
}
```



Partition Example



- Partition

  Time complexity of partition is $\Theta(n) + g(n)$, where $g(n)$ is time taken by choosePivot.

- Q: How can we choose "good" pivots "fast"?

- Quicksort is the most-used sorting algorithm in practise, so there must be a way.

- But: - what qualifies as "fast"?

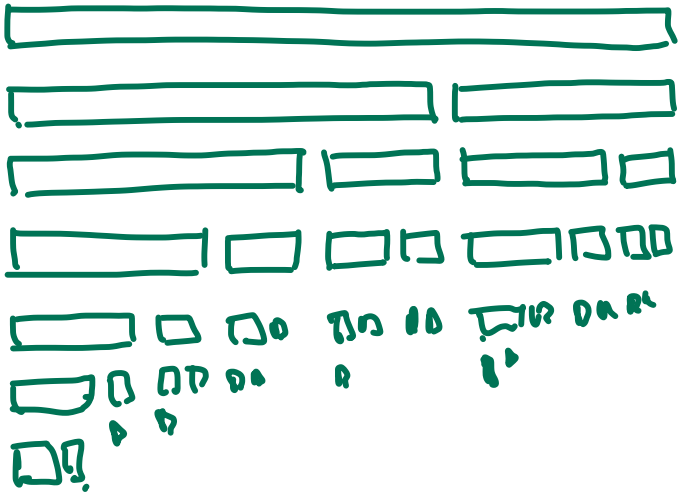
  probably a very small constant

  - what qualifies as "good"?

  (given that it must be fast)

Consider:

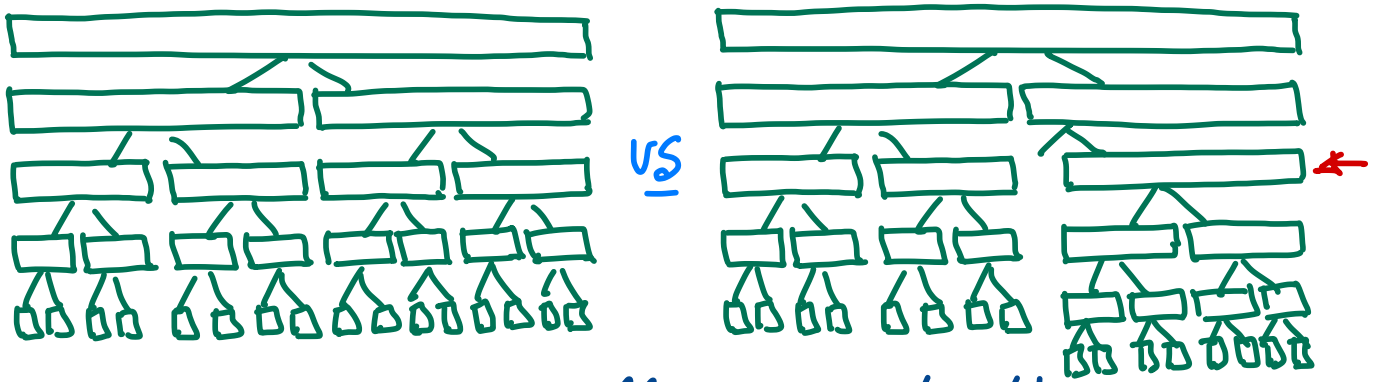
- Perfect pivots are not needed for $O(n \log n)$ time.



Eg. if pivots are all better than $\frac{1}{3} : \frac{2}{3}$, then depth is $\log_{\frac{3}{2}} n$

So we still get $O(n \log n)$.

- A small number of bad pivots:



makes a small difference in height.

Some "simple" choosePivot options

- A[hi] - fast, but performs badly on many inputs.
- median - perfect pivots, but too slow to compute
- random:
  - If pivots are chosen uniformly at random, then Quicksort runs in time $O(n \log n)$ with probability $1 - 1/2^n$ - ie almost always.
  - But: good random numbers are not fast to make.
- median{ A[lo], A[hi], A[(hi+lo)/2] }
  - fast
  - not very easy to come up with a "very bad" input. // so, maybe they will be infrequent in practice

Complexity of Quicksort

- Depends critically on how pivots are chosen
- Choosing "perfect" pivots is too slow for a practical sorting algorithm
- Fortunately, choosing pivots that are "good enough" for most inputs can be done fast.
- Quicksort – with practical pivot choice strategies – has a $\Theta(n^2)$ worst case time, but is often described as "like $\Theta(n \log n)$ in practice".

- $O(n \log n)$ average case is often mentioned, and often confused with "typical in practice" a "almost always in practise" – but average-case analyses often have nothing to do with practical cases.

End

CPU Cache & Sorting:

Insertion Sort:

repeated sequential scans of array prefixes

Selection Sort:

repeated sequential scans of array suffixes

Heapsort:

repeated percolate-downs in array prefixes

Merge Sort:

repeated sequential scans of segments of two arrays.

Quicksort:

repeated sequential scans of sub-arrays.

In practice

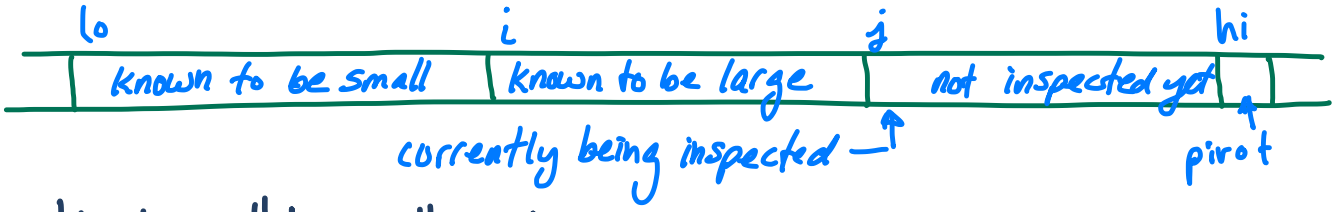
- In most settings, the preferred algorithm is Quicksort
- There are settings where Mergesort & Insertion Sort are preferred
- For small sets, Selection Sort is faster
- Often, this variant (or similar) is faster:

```
quicksort(A, lo, hi){
    if(lo < hi){ // there are ≥ 2 items
        if(hi - lo < 15){ // less than 15 items
            selectionSort(A, lo, hi) // OR insertion sort, OR...
        }
        else{
            pivotposition ← partition(A, lo, hi) // partition
            quicksort(A, lo, pivotposition - 1)
            quicksort(A, pivotposition + 1, hi)
        }
    }
}
```

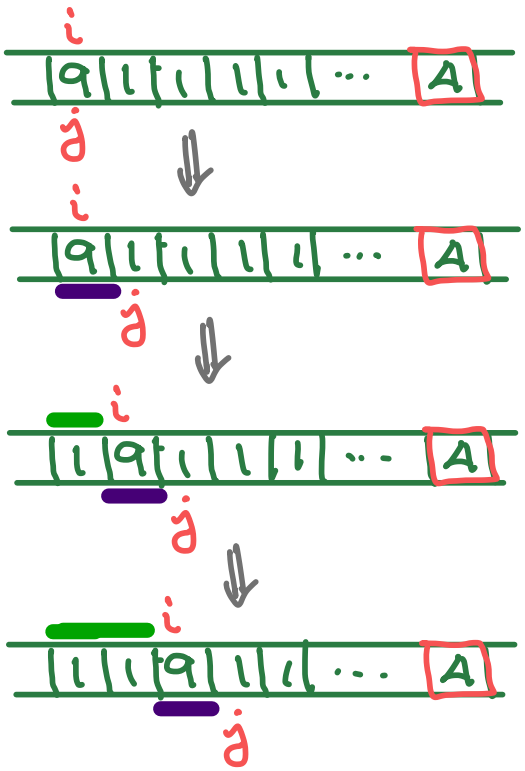
- There is a faster version of partition.

Partition Improvement 1 - Two-way search

- Simple partition has "known part" divided into "small" + "large"



- notice that a "large" value might get (pointlessly) moved multiple times:



- Improvement: Keep "known large" at high indexes.

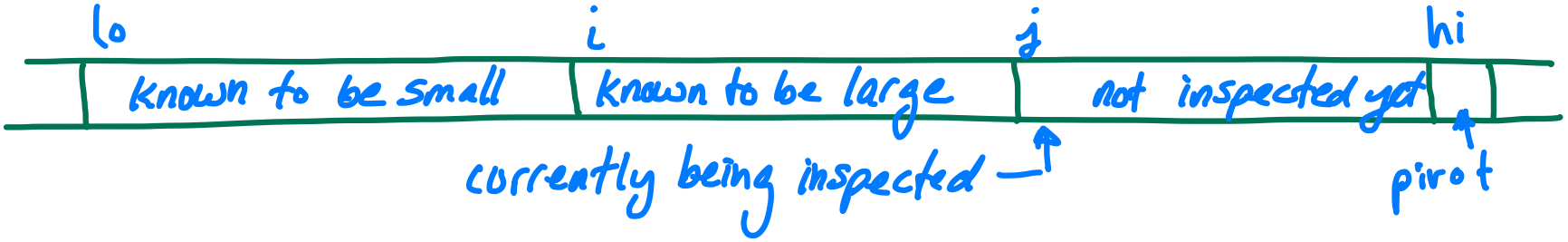
Partition With 2-Sided Search


```
partition(A, lo, hi){
    pivotindex = choosePivot(A, lo, hi)
    swap A[pivotindex] & A[hi]
    p = A[hi]
    L = low
    r = hi - 1
    repeat{
      while (A[lo] < p) L++;
      while(A[r] ≥ p) r--;
      if( L < r){
            swap A[r] & A[L]
            L++
            r--
      }
    }until( L ≥ r)
    swap A[L] and A[hi]
}
```

swap A[i] and A[hi] // move pivot where it belongs.
return i // this is pivot position

Partition Improvement 1 – Tri-partition

- Simple partition has "known part" divided into "small" + "large"



  - If there are multiple occurrences of the pivot, they are considered "large" – and get sent to a recursive quicksort call.

- Improved: keep all pivot values together, and leave out of recursion.

Merge Sort on Lists

- Consider data in lists not arrays – i.e. any sequential access device or data structures:
  - linked list
  - disk, tape, ... storage